

Bachelor of Technology

Second Year – Semester III

DBMS Lab

Course Code: U25AMILPC307

PRACTICAL P3

DML Operations and Data Retrieval – INSERT, UPDATE, DELETE, and Filtering using WHERE, ORDER BY and LIMIT

Database: *college_db* (College Management System) | PostgreSQL 17

Name of Student	Sagar Kumar
PRN	SOE25BTAM29
Section	A
Program	Bachelor of Technology
Year / Semester	2 / III
Course Code	U25AMILPC307
Course Title	DBMS Lab
Practical No.	P3
Date of Performance	
Date of Submission	

Signature of Student

Signature of Faculty

1. Aim

To perform Data Manipulation Language (DML) operations – **INSERT**, **UPDATE** and **DELETE** – on the College Management System database *college_db* created in Practicals P1 and P2, and to retrieve data using **SELECT** with filtering and sorting through the **WHERE**, **ORDER BY** and **LIMIT** clauses in PostgreSQL 17.

2. Objective

- To populate the tables *department*, *faculty*, *student*, *course* and *enrollment* with meaningful sample data using single and multi-row **INSERT INTO** statements.
- To display complete table contents using **SELECT *** and selected columns using a column list.
- To filter rows with the **WHERE** clause using comparison (**=**, **<>**, **>=**), logical (**AND**), range (**BETWEEN**) and pattern (**LIKE**) operators.
- To sort result sets with **ORDER BY** in ascending and descending order and on multiple columns.
- To restrict the number of rows returned using **LIMIT** and **OFFSET** (Top-N queries and paging).
- To modify existing records with **UPDATE** and remove records with **DELETE**, and verify every change with a **SELECT**.
- To observe how the FOREIGN KEY constraints defined in P2 protect referential integrity during **DELETE**.

3. Theory

3.1 Data Manipulation Language (DML)

SQL commands are grouped into DDL (**CREATE**, **ALTER**, **DROP**), DML (**INSERT**, **UPDATE**, **DELETE**), DQL (**SELECT**), DCL (**GRANT**, **REVOKE**) and TCL (**COMMIT**, **ROLLBACK**). DDL defines the *structure* of the database, which was done in P1 and P2. DML works on the *data* stored inside that structure – it adds, changes and removes rows without altering the table definition. **SELECT** is often grouped with DML because it is the statement used to read the data that DML writes.

3.2 INSERT INTO

INSERT adds new rows to a table. Listing the column names is good practice because the statement then does not depend on the physical order of columns. PostgreSQL allows several rows in one statement; the whole statement succeeds or fails as one unit. Every value is checked against the column's data type and the **PRIMARY KEY**, **UNIQUE**, **NOT NULL**, **CHECK** and **FOREIGN KEY** constraints. On success `psql` prints **INSERT 0 n**, where *n* is the number of rows inserted (the 0 is a legacy OID field).

```
INSERT INTO table_name (col1, col2, ...) VALUES (v1, v2, ...), (v1, v2, ...);
```

3.3 SELECT and the WHERE clause

SELECT retrieves rows. **SELECT *** returns all columns; a column list returns only the required columns (projection). The **WHERE** clause keeps only the rows for which the condition is **TRUE** (selection).

Operator type	Operators	Example used in this practical
Comparison	= , <> , < , > , <= , >=	WHERE marks >= 85

Operator type	Operators	Example used in this practical
Logical	AND, OR, NOT	WHERE city = 'Pune' AND gender = 'F'
Range (inclusive)	BETWEEN a AND b	WHERE marks BETWEEN 60 AND 75
Pattern	LIKE / ILIKE with % and _	WHERE course_name LIKE '%Systems'
Set membership	IN (...)	WHERE dept_id IN (101, 102)
Null test	IS NULL / IS NOT NULL	WHERE grade IS NULL

```
SELECT col1, col2 FROM table_name WHERE condition;
```

3.4 ORDER BY

Rows in a relational table have no guaranteed order, so ORDER BY is the only way to get a predictable sequence. ASC (default) sorts ascending and DESC sorts descending. When more than one column is given, the second column decides the order only among rows that have equal values in the first. In PostgreSQL NULLs sort last in ASC and first in DESC unless NULLS FIRST/LAST is written.

```
SELECT ... FROM table_name ORDER BY col1 ASC, col2 DESC;
```

3.5 LIMIT and OFFSET

LIMIT n returns at most n rows; OFFSET m skips the first m rows. They are applied after ORDER BY, so LIMIT must be combined with ORDER BY to obtain meaningful results such as “top 3 marks”. LIMIT/OFFSET together are used for paging (page k of size n uses OFFSET (k-1)×n).

```
SELECT ... FROM table_name ORDER BY col DESC LIMIT n OFFSET m;
```

3.6 UPDATE

UPDATE changes column values of existing rows. The SET list may contain constants or expressions based on the current value (for example marks = marks + 3). Only the rows matching the WHERE clause are changed; **without WHERE every row in the table is updated**. psql reports UPDATE n with the number of rows affected.

```
UPDATE table_name SET col1 = value1, col2 = expr WHERE condition;
```

3.7 DELETE

DELETE removes rows that satisfy the WHERE condition; without WHERE it removes all rows but keeps the table structure. If a row is still referenced by a FOREIGN KEY in another table, PostgreSQL rejects the delete (default ON DELETE NO ACTION) so that no orphan child rows are left. psql reports DELETE n.

```
DELETE FROM table_name WHERE condition;
```

In psql every statement runs in *autocommit* mode, so each successful DML statement is committed immediately; wrapping statements in BEGIN ... COMMIT/ROLLBACK makes them a single transaction.

4. Database Description

The practical uses the same **College Management System** database, **college_db**, designed in P1 and P2. It contains five related tables. A department has many faculty members, students and courses; each course is taught by one faculty member; and the *enrollment* table resolves the many-to-many relationship between *student* and *course*, storing the marks and grade obtained.

Table	Primary key	Foreign keys	Purpose
department	dept_id	–	Academic departments
faculty	faculty_id	dept_id → department	Teaching staff
student	student_id	dept_id → department	Registered students (PRN is UNIQUE)
course	course_code	dept_id → department, faculty_id → faculty	Courses offered in Semester III
enrollment	enrollment_id	student_id → student, course_code → course	Student–course registrations with marks and grade

Grading scheme used for the *grade* column: O (90–100), A+ (80–89), A (70–79), B+ (60–69), B (50–59), C (40–49), F (below 40).

Table structure (as created in P1/P2):

```
CREATE TABLE department (  
    dept_id      INT          PRIMARY KEY,  
    dept_name    VARCHAR(60) NOT NULL UNIQUE,  
    hod_name     VARCHAR(60),  
    building     VARCHAR(40)  
);  
  
CREATE TABLE faculty (  
    faculty_id   INT          PRIMARY KEY,  
    faculty_name VARCHAR(60) NOT NULL,  
    designation  VARCHAR(40),  
    email        VARCHAR(80) UNIQUE,  
    dept_id     INT          REFERENCES department(dept_id)  
);  
  
CREATE TABLE student (  
    student_id   INT          PRIMARY KEY,  
    prn          VARCHAR(15) NOT NULL UNIQUE,  
    student_name VARCHAR(60) NOT NULL,  
    gender       CHAR(1)      CHECK (gender IN ('M','F')),  
    dob          DATE,  
    email        VARCHAR(80) UNIQUE,  
    city         VARCHAR(40),  
    semester     INT          CHECK (semester BETWEEN 1 AND 8),  
    dept_id     INT          REFERENCES department(dept_id)  
);  
  
CREATE TABLE course (  
    course_code  VARCHAR(10) PRIMARY KEY,  
    course_name  VARCHAR(80) NOT NULL,  
    credits      INT          CHECK (credits BETWEEN 1 AND 6),  
    semester     INT,  
    dept_id     INT          REFERENCES department(dept_id),  
    faculty_id  INT          REFERENCES faculty(faculty_id)  
);  
  
CREATE TABLE enrollment (  
    enrollment_id INT          PRIMARY KEY,  
    student_id   INT          NOT NULL REFERENCES student(student_id),  
    course_code   VARCHAR(10) NOT NULL REFERENCES course(course_code),  
    enrollment_date DATE       DEFAULT CURRENT_DATE,  
    marks         NUMERIC(5,2) CHECK (marks BETWEEN 0 AND 100),  
    grade        VARCHAR(2),  
    UNIQUE (student_id, course_code)  
);
```

5. SQL Queries

The queries below were executed in the order shown, in one psql session connected to *college_db*. Each group corresponds to one screenshot in Section 7.

5.1 Connect to the database and verify the tables from P1/P2

Shell command used to start the session: `psql -U postgres -d college_db`

```
SELECT version();
\dt
```

5.2 INSERT – department, faculty, student

```
INSERT INTO department (dept_id, dept_name, hod_name, building) VALUES
(101, 'Computer Science', 'Dr. Anil Sharma', 'Block A'),
(102, 'Artificial Intelligence', 'Dr. Kavita Mehta', 'Block C'),
(103, 'Electronics', 'Dr. Rajesh Patil', 'Block B');

INSERT INTO faculty (faculty_id, faculty_name, designation, email, dept_id) VALUES
(201, 'Dr. Anil Sharma', 'Professor', 'anil.sharma@college.edu', 101),
(202, 'Dr. Kavita Mehta', 'Associate Professor', 'kavita.mehta@college.edu', 102),
(203, 'Dr. Rajesh Patil', 'Assistant Professor', 'rajesh.patil@college.edu', 103);

INSERT INTO student (student_id, prn, student_name, gender, dob, email, city, semester, dept_id) VALUES
(1, 'SOE25BTAM29', 'Sagar Kumar', 'M', '2006-08-14', 'sagar.kumar@college.edu', 'Pune', 3, 102),
(2, 'SOE25BTCS07', 'Rahul Sharma', 'M', '2006-03-22', 'rahul.sharma@college.edu', 'Nashik', 3, 101),
(3, 'SOE25BTAM41', 'Priya Singh', 'F', '2006-11-05', 'priya.singh@college.edu', 'Nagpur', 3, 102),
(4, 'SOE25BTCEC15', 'Aman Verma', 'M', '2005-12-19', 'aman.verma@college.edu', 'Indore', 3, 103),
(5, 'SOE25BTCS33', 'Neha Joshi', 'F', '2006-06-30', 'neha.joshi@college.edu', 'Pune', 3, 101);
```

5.3 INSERT – course, enrollment

```
INSERT INTO course (course_code, course_name, credits, semester, dept_id, faculty_id) VALUES
('CS201', 'Data Structures', 4, 3, 101, 201),
('CS202', 'Database Management Systems', 4, 3, 101, 201),
('CS203', 'Operating Systems', 3, 3, 101, 203),
('AI201', 'Artificial Intelligence', 3, 3, 102, 202);

INSERT INTO enrollment (enrollment_id, student_id, course_code, enrollment_date, marks, grade) VALUES
(1, 1, 'CS202', '2025-07-21', 86, 'A+'),
(2, 1, 'CS201', '2025-07-21', 78, 'A'),
(3, 1, 'AI201', '2025-07-21', 91, 'O'),
(4, 2, 'CS202', '2025-07-21', 72, 'A'),
(5, 2, 'CS201', '2025-07-21', 65, 'B+'),
(6, 2, 'CS203', '2025-07-22', 58, 'B'),
(7, 3, 'CS202', '2025-07-21', 94, 'O'),
(8, 3, 'AI201', '2025-07-21', 88, 'A+'),
(9, 4, 'CS201', '2025-07-22', 47, 'C'),
(10, 4, 'CS203', '2025-07-22', 62, 'B+'),
(11, 5, 'CS202', '2025-07-21', 68, 'B+'),
(12, 5, 'CS203', '2025-07-22', 81, 'A+');
```

5.4 SELECT * – department, faculty, student

```
SELECT * FROM department;
SELECT * FROM faculty;
SELECT * FROM student;
```

5.5 SELECT * – course, enrollment

```
SELECT * FROM course;
SELECT * FROM enrollment;
```

5.6 SELECT with WHERE

```
SELECT student_id, prn, student_name, city FROM student WHERE dept_id = 102;
SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment WHERE marks >= 85;
SELECT course_code, course_name, credits FROM course WHERE course_name LIKE '%Systems';
SELECT student_id, course_code, marks FROM enrollment WHERE marks BETWEEN 60 AND 75
    AND course_code <> 'CS202';
SELECT student_name, city FROM student WHERE city = 'Pune' AND gender = 'F';
```

5.7 SELECT with ORDER BY

```
SELECT student_id, student_name, city FROM student ORDER BY student_name ASC;
SELECT student_id, course_code, marks, grade FROM enrollment WHERE course_code = 'CS202'
    ORDER BY marks DESC;
SELECT student_name, dept_id, dob FROM student ORDER BY dept_id ASC, dob DESC;
```

5.8 SELECT with LIMIT / OFFSET

```
SELECT enrollment_id, student_id, course_code, marks FROM enrollment ORDER BY marks DESC LIMIT 3;
SELECT enrollment_id, student_id, course_code, marks FROM enrollment ORDER BY marks DESC
    LIMIT 3 OFFSET 3;
SELECT student_name, dob FROM student ORDER BY dob ASC LIMIT 1;
```

5.9 UPDATE – re-evaluation of marks (with verification)

Neha Joshi's DBMS (CS202) answer sheet was re-evaluated: marks 68 → 72, grade B+ → A.

```
SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment
    WHERE student_id = 5 AND course_code = 'CS202';
UPDATE enrollment SET marks = 72, grade = 'A' WHERE student_id = 5 AND course_code = 'CS202';
SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment
    WHERE student_id = 5 AND course_code = 'CS202';
```

5.10 UPDATE – expression-based update and text column update (with verification)

Grace of 3 marks to students scoring below 50 in Data Structures (CS201); Rahul Sharma changed his city to Mumbai.

```
UPDATE enrollment SET marks = marks + 3, grade = 'B' WHERE course_code = 'CS201' AND marks < 50;
SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment
    WHERE course_code = 'CS201' ORDER BY student_id;
UPDATE student SET city = 'Mumbai' WHERE prn = 'SOE25BTCS07';
SELECT student_id, prn, student_name, city FROM student WHERE prn = 'SOE25BTCS07';
```

5.11 DELETE – withdrawal from a course (with verification)

Aman Verma withdrew from Operating Systems (CS203).

```
SELECT enrollment_id, student_id, course_code, marks FROM enrollment WHERE student_id = 4;
DELETE FROM enrollment WHERE student_id = 4 AND course_code = 'CS203';
SELECT enrollment_id, student_id, course_code, marks FROM enrollment WHERE student_id = 4;
SELECT COUNT(*) AS total_enrollments FROM enrollment;
```

5.12 DELETE on a referenced row, and final state of enrollment

Attempt to delete Aman Verma from *student* while he still has an enrollment row.

```
DELETE FROM student WHERE student_id = 4;
SELECT * FROM enrollment ORDER BY enrollment_id;
```

6. Execution

Item	Details
DBMS	PostgreSQL 17.10 (server)
Client	psql 17.10 – PostgreSQL interactive terminal (Linux, x86_64)
Database	college_db (schema from P1/P2, tables initially empty)
Mode	Interactive psql session, autocommit ON, pager OFF

Procedure:

1. Started the PostgreSQL 17 server and opened a terminal.
2. Connected to the database with `psql -U postgres -d college_db` and confirmed the server version and the five tables using `SELECT version();` and `\dt`.
3. Typed the INSERT statements for the parent tables first (department, faculty, student) and then for the child tables (course, enrollment), so that every FOREIGN KEY value already existed.
4. Displayed all tables using `SELECT *` to verify the inserted data.
5. Executed the WHERE, ORDER BY and LIMIT/OFFSET queries.
6. Executed the UPDATE statements, each followed by a SELECT to verify the change.
7. Executed the DELETE statements, each followed by a SELECT to verify the change, and displayed the final enrollment table.
8. Captured a screenshot of the terminal after each group of statements (Section 7) and exited psql with `\q`.

7. Output

The following screenshots were taken from the psql terminal while the queries of Section 5 were being executed. They are shown in execution order and have not been edited apart from cropping the empty part of the terminal window.

```
$ psql -U postgres -d college_db
psql (17.10)
Type "help" for help.

college_db=# SELECT version();

               version
-----
PostgreSQL 17.10 on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 7.5.0-3ubuntu1-18.04) 7.5.0, 64-bit
(1 row)

college_db=# \dt
      List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | course         | table | postgres
public | department     | table | postgres
public | enrollment     | table | postgres
public | faculty        | table | postgres
public | student        | table | postgres
(5 rows)

college_db=#
```

Screenshot 1: Connecting to college_db and listing the tables created in P1/P2

```
college_db=# INSERT INTO department (dept_id, dept_name, hod_name, building) VALUES
college_db-# (101, 'Computer Science', 'Dr. Anil Sharma', 'Block A'),
college_db-# (102, 'Artificial Intelligence', 'Dr. Kavita Mehta', 'Block C'),
college_db-# (103, 'Electronics', 'Dr. Rajesh Patil', 'Block B');
INSERT 0 3
college_db=# INSERT INTO faculty (faculty_id, faculty_name, designation, email, dept_id) VALUES
college_db-# (201, 'Dr. Anil Sharma', 'Professor', 'anil.sharma@college.edu', 101),
college_db-# (202, 'Dr. Kavita Mehta', 'Associate Professor', 'kavita.mehta@college.edu', 102),
college_db-# (203, 'Dr. Rajesh Patil', 'Assistant Professor', 'rajesh.patil@college.edu', 103);
INSERT 0 3
college_db=# INSERT INTO student (student_id, prn, student_name, gender, dob, email, city, semester, dept_id) VALUES
college_db-# (1, 'SOE25BTAM29', 'Sagar Kumar', 'M', '2006-08-14', 'sagar.kumar@college.edu', 'Pune', 3, 102),
college_db-# (2, 'SOE25BTCS07', 'Rahul Sharma', 'M', '2006-03-22', 'rahul.sharma@college.edu', 'Nashik', 3, 101),
college_db-# (3, 'SOE25BTAM41', 'Priya Singh', 'F', '2006-11-05', 'priya.singh@college.edu', 'Nagpur', 3, 102),
college_db-# (4, 'SOE25BTEC15', 'Aman Verma', 'M', '2005-12-19', 'aman.verma@college.edu', 'Indore', 3, 103),
college_db-# (5, 'SOE25BTCS33', 'Neha Joshi', 'F', '2006-06-30', 'neha.joshi@college.edu', 'Pune', 3, 101);
INSERT 0 5
college_db=#
```

Screenshot 2: INSERT: department, faculty and student tables

```
college_db=# INSERT INTO course (course_code, course_name, credits, semester, dept_id, faculty_id) VALUES
college_db-# ('CS201', 'Data Structures', 4, 3, 101, 201),
college_db-# ('CS202', 'Database Management Systems', 4, 3, 101, 201),
college_db-# ('CS203', 'Operating Systems', 3, 3, 101, 203),
college_db-# ('AI201', 'Artificial Intelligence', 3, 3, 102, 202);
INSERT 0 4
college_db=# INSERT INTO enrollment (enrollment_id, student_id, course_code, enrollment_date, marks, grade) VALUES
college_db-# (1, 1, 'CS202', '2025-07-21', 86, 'A+'),
college_db-# (2, 1, 'CS201', '2025-07-21', 78, 'A'),
college_db-# (3, 1, 'AI201', '2025-07-21', 91, 'O'),
college_db-# (4, 2, 'CS201', '2025-07-21', 72, 'A'),
college_db-# (5, 2, 'CS201', '2025-07-21', 65, 'B+'),
college_db-# (6, 2, 'CS203', '2025-07-22', 58, 'B'),
college_db-# (7, 3, 'CS202', '2025-07-21', 94, 'O'),
college_db-# (8, 3, 'AI201', '2025-07-21', 88, 'A+'),
college_db-# (9, 4, 'CS201', '2025-07-22', 47, 'C'),
college_db-# (10, 4, 'CS203', '2025-07-22', 62, 'B+'),
college_db-# (11, 5, 'CS202', '2025-07-21', 68, 'B+'),
college_db-# (12, 5, 'CS203', '2025-07-22', 81, 'A+');
INSERT 0 12
college_db=#
```

Screenshot 3: INSERT: course and enrollment tables

```
college_db=# SELECT * FROM department;
dept_id | dept_name | hod_name | building
-----+-----+-----+-----
    101 | Computer Science | Dr. Anil Sharma | Block A
    102 | Artificial Intelligence | Dr. Kavita Mehta | Block C
    103 | Electronics | Dr. Rajesh Patil | Block B
(3 rows)

college_db=# SELECT * FROM faculty;
faculty_id | faculty_name | designation | email | dept_id
-----+-----+-----+-----+-----
    201 | Dr. Anil Sharma | Professor | anil.sharma@college.edu | 101
    202 | Dr. Kavita Mehta | Associate Professor | kavita.mehta@college.edu | 102
    203 | Dr. Rajesh Patil | Assistant Professor | rajesh.patil@college.edu | 103
(3 rows)

college_db=# SELECT * FROM student;
student_id | prn | student_name | gender | dob | email | city | semester | dept_id
-----+-----+-----+-----+-----+-----+-----+-----+-----
    1 | SOE25BTAM29 | Sagar Kumar | M | 2006-08-14 | sagar.kumar@college.edu | Pune | 3 | 102
    2 | SOE25BTC507 | Rahul Sharma | M | 2006-03-22 | rahul.sharma@college.edu | Nashik | 3 | 101
    3 | SOE25BTAM41 | Priya Singh | F | 2006-11-05 | priya.singh@college.edu | Nagpur | 3 | 102
    4 | SOE25BTEC15 | Aman Verma | M | 2005-12-19 | aman.verma@college.edu | Indore | 3 | 103
    5 | SOE25BTC533 | Neha Joshi | F | 2006-06-30 | neha.joshi@college.edu | Pune | 3 | 101
(5 rows)

college_db=# █
```

Screenshot 4: SELECT *: department, faculty and student

```
college_db=# SELECT * FROM course;
course_code | course_name | credits | semester | dept_id | faculty_id
-----+-----+-----+-----+-----+-----
CS201 | Data Structures | 4 | 3 | 101 | 201
CS202 | Database Management Systems | 4 | 3 | 101 | 201
CS203 | Operating Systems | 3 | 3 | 101 | 203
AI201 | Artificial Intelligence | 3 | 3 | 102 | 202
(4 rows)

college_db=# SELECT * FROM enrollment;
enrollment_id | student_id | course_code | enrollment_date | marks | grade
-----+-----+-----+-----+-----+-----
    1 | 1 | CS202 | 2025-07-21 | 86.00 | A+
    2 | 1 | CS201 | 2025-07-21 | 78.00 | A
    3 | 1 | AI201 | 2025-07-21 | 91.00 | O
    4 | 2 | CS202 | 2025-07-21 | 72.00 | A
    5 | 2 | CS201 | 2025-07-21 | 65.00 | B+
    6 | 2 | CS203 | 2025-07-22 | 58.00 | B
    7 | 3 | CS202 | 2025-07-21 | 94.00 | O
    8 | 3 | AI201 | 2025-07-21 | 88.00 | A+
    9 | 4 | CS201 | 2025-07-22 | 47.00 | C
   10 | 4 | CS203 | 2025-07-22 | 62.00 | B+
   11 | 5 | CS202 | 2025-07-21 | 68.00 | B+
   12 | 5 | CS203 | 2025-07-22 | 81.00 | A+
(12 rows)

college_db=# █
```

Screenshot 5: SELECT *: course and enrollment

```
college_db=# SELECT student_id, prn, student_name, city FROM student WHERE dept_id = 102;
student_id | prn      | student_name | city
-----+-----+-----+-----
          1 | SOE25BTAM29 | Sagar Kumar  | Pune
          3 | SOE25BTAM41 | Priya Singh  | Nagpur
(2 rows)

college_db=# SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment WHERE marks >= 85;
enrollment_id | student_id | course_code | marks | grade
-----+-----+-----+-----+-----
          1 |          1 | CS202      | 86.00 | A+
          3 |          1 | AI201      | 91.00 | 0
          7 |          3 | CS202      | 94.00 | 0
          8 |          3 | AI201      | 88.00 | A+
(4 rows)

college_db=# SELECT course_code, course_name, credits FROM course WHERE course_name LIKE '%Systems';
course_code | course_name          | credits
-----+-----+-----
CS202      | Database Management Systems | 4
CS203      | Operating Systems      | 3
(2 rows)

college_db=# SELECT student_id, course_code, marks FROM enrollment WHERE marks BETWEEN 60 AND 75 AND course_code <> 'CS202';
student_id | course_code | marks
-----+-----+-----
          2 | CS201      | 65.00
          4 | CS203      | 62.00
(2 rows)

college_db=# SELECT student_name, city FROM student WHERE city = 'Pune' AND gender = 'F';
student_name | city
-----+-----
Neha Joshi   | Pune
(1 row)

college_db=# █
```

Screenshot 6: SELECT with WHERE (=, >=, LIKE, BETWEEN, AND)

```
college_db=# SELECT student_id, student_name, city FROM student ORDER BY student_name ASC;
student_id | student_name | city
-----+-----+-----
          4 | Aman Verma   | Indore
          5 | Neha Joshi   | Pune
          3 | Priya Singh  | Nagpur
          2 | Rahul Sharma | Nashik
          1 | Sagar Kumar  | Pune
(5 rows)

college_db=# SELECT student_id, course_code, marks, grade FROM enrollment WHERE course_code = 'CS202' ORDER BY marks DESC;
student_id | course_code | marks | grade
-----+-----+-----+-----
          3 | CS202      | 94.00 | 0
          1 | CS202      | 86.00 | A+
          2 | CS202      | 72.00 | A
          5 | CS202      | 68.00 | B+
(4 rows)

college_db=# SELECT student_name, dept_id, dob FROM student ORDER BY dept_id ASC, dob DESC;
student_name | dept_id | dob
-----+-----+-----
Neha Joshi   | 101    | 2006-06-30
Rahul Sharma | 101    | 2006-03-22
Priya Singh  | 102    | 2006-11-05
Sagar Kumar  | 102    | 2006-08-14
Aman Verma   | 103    | 2005-12-19
(5 rows)

college_db=# █
```

Screenshot 7: SELECT with ORDER BY (ASC, DESC, multiple columns)

```
college_db=# SELECT enrollment_id, student_id, course_code, marks FROM enrollment ORDER BY marks DESC LIMIT 3;
enrollment_id | student_id | course_code | marks
-----+-----+-----+-----
          7 |          3 | CS202      | 94.00
          3 |          1 | AI201      | 91.00
          8 |          3 | AI201      | 88.00
(3 rows)

college_db=# SELECT enrollment_id, student_id, course_code, marks FROM enrollment ORDER BY marks DESC LIMIT 3 OFFSET 3;
enrollment_id | student_id | course_code | marks
-----+-----+-----+-----
          1 |          1 | CS202      | 86.00
         12 |          5 | CS203      | 81.00
          2 |          1 | CS201      | 78.00
(3 rows)

college_db=# SELECT student_name, dob FROM student ORDER BY dob ASC LIMIT 1;
student_name | dob
-----+-----
Aman Verma   | 2005-12-19
(1 row)

college_db=# █
```

Screenshot 8: SELECT with LIMIT and OFFSET

```
college_db=# SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment WHERE student_id = 5 AND course_code = 'CS202';
enrollment_id | student_id | course_code | marks | grade
-----+-----+-----+-----+-----
          11 |          5 | CS202      | 68.00 | B+
(1 row)

college_db=# UPDATE enrollment SET marks = 72, grade = 'A' WHERE student_id = 5 AND course_code = 'CS202';
UPDATE 1
college_db=# SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment WHERE student_id = 5 AND course_code = 'CS202';
enrollment_id | student_id | course_code | marks | grade
-----+-----+-----+-----+-----
          11 |          5 | CS202      | 72.00 | A
(1 row)

college_db=# █
```

Screenshot 9: UPDATE a single row (re-evaluation) with verification

```
college_db=# UPDATE enrollment SET marks = marks + 3, grade = 'B' WHERE course_code = 'CS201' AND marks < 50;
UPDATE 1
college_db=# SELECT enrollment_id, student_id, course_code, marks, grade FROM enrollment WHERE course_code = 'CS201' ORDER BY student_id;
enrollment_id | student_id | course_code | marks | grade
-----+-----+-----+-----+-----
          2 |          1 | CS201      | 78.00 | A
          5 |          2 | CS201      | 65.00 | B+
          9 |          4 | CS201      | 50.00 | B
(3 rows)

college_db=# UPDATE student SET city = 'Mumbai' WHERE prn = 'SOE25BTC507';
UPDATE 1
college_db=# SELECT student_id, prn, student_name, city FROM student WHERE prn = 'SOE25BTC507';
student_id | prn      | student_name | city
-----+-----+-----+-----
          2 | SOE25BTC507 | Rahul Sharma | Mumbai
(1 row)

college_db=# █
```

Screenshot 10: UPDATE using an expression and UPDATE of a text column, with verification

```
college_db=# SELECT enrollment_id, student_id, course_code, marks FROM enrollment WHERE student_id = 4;
enrollment_id | student_id | course_code | marks
-----+-----+-----+-----
          9 |         4 | CS201      |  50.00
         10 |         4 | CS203      |  62.00
(2 rows)

college_db=# DELETE FROM enrollment WHERE student_id = 4 AND course_code = 'CS203';
DELETE 1
college_db=# SELECT enrollment_id, student_id, course_code, marks FROM enrollment WHERE student_id = 4;
enrollment_id | student_id | course_code | marks
-----+-----+-----+-----
          9 |         4 | CS201      |  50.00
(1 row)

college_db=# SELECT COUNT(*) AS total_enrollments FROM enrollment;
total_enrollments
-----
          11
(1 row)

college_db=# █
```

Screenshot 11: DELETE with WHERE and verification

```
college_db=# DELETE FROM student WHERE student_id = 4;
ERROR:  update or delete on table "student" violates foreign key constraint "enrollment_student_id_fkey" on table "enrollment"
DETAIL:  Key (student_id)=(4) is still referenced from table "enrollment".
college_db=# SELECT * FROM enrollment ORDER BY enrollment_id;
enrollment_id | student_id | course_code | enrollment_date | marks | grade
-----+-----+-----+-----+-----+-----
          1 |         1 | CS202      | 2025-07-21      |  86.00 | A+
          2 |         1 | CS201      | 2025-07-21      |  78.00 | A
          3 |         1 | AI201      | 2025-07-21      |  91.00 | O
          4 |         2 | CS202      | 2025-07-21      |  72.00 | A
          5 |         2 | CS201      | 2025-07-21      |  65.00 | B+
          6 |         2 | CS203      | 2025-07-22      |  58.00 | B
          7 |         3 | CS202      | 2025-07-21      |  94.00 | O
          8 |         3 | AI201      | 2025-07-21      |  88.00 | A+
          9 |         4 | CS201      | 2025-07-22      |  50.00 | B
         11 |         5 | CS202      | 2025-07-21      |  72.00 | A
         12 |         5 | CS203      | 2025-07-22      |  81.00 | A+
(11 rows)

college_db=# █
```

Screenshot 12: DELETE blocked by FOREIGN KEY, and final state of enrollment

8. Explanation of Results

Connection (Screenshot 1): SELECT version() confirms PostgreSQL 17.10, and \dt lists exactly the five tables of P1/P2 – course, department, enrollment, faculty and student – so the practical runs on the same database.

INSERT (Screenshots 2–3): psql replied INSERT 0 3 (department), INSERT 0 3 (faculty), INSERT 0 5 (student), INSERT 0 4 (course) and INSERT 0 12 (enrollment): 27 rows in total, with no constraint violation. Inserting the parent tables before the child tables satisfied all FOREIGN KEY constraints.

SELECT * (Screenshots 4–5): Each table returned exactly the rows inserted – 3, 3, 5, 4 and 12 rows. NUMERIC(5,2) values such as 86 are displayed as 86.00, and dates are shown in ISO format (YYYY-MM-DD).

WHERE (Screenshot 6): dept_id = 102 returned the two Artificial Intelligence students, Sagar Kumar and Priya Singh. marks >= 85 returned 4 enrollments (86, 91, 94, 88). LIKE '%Systems' matched only course names ending in “Systems”: Database Management Systems and Operating Systems. BETWEEN 60 AND 75 AND course_code <> 'CS202' returned 2 rows (65.00 and 62.00); the CS202 marks 72 and 68 were excluded by the second condition. The AND condition on city and gender returned only Neha Joshi.

ORDER BY (Screenshot 7): Names were sorted alphabetically from Aman Verma to Sagar Kumar. The DBMS (CS202) marks were listed in descending order 94, 86, 72, 68, giving a rank list. In the two-column sort, students were grouped by dept_id 101, 102, 103 and, inside each department, the younger student (later date of birth) appears first because of `dob DESC`.

LIMIT / OFFSET (Screenshot 8): `LIMIT 3` after sorting by marks `DESC` gave the top three scores (94, 91, 88). `LIMIT 3 OFFSET 3` skipped those and returned the next three (86, 81, 78) – the second “page”. `ORDER BY dob ASC LIMIT 1` returned the oldest student, Aman Verma (2005-12-19).

UPDATE (Screenshots 9–10): Each `UPDATE` reported `UPDATE 1`, i.e. exactly one row matched the `WHERE` clause. The verification `SELECT`s show Neha Joshi's CS202 row changed from 68.00/B+ to 72.00/A; Aman Verma's CS201 marks increased by the expression `marks + 3` from 47 to 50.00 with grade B, while the other CS201 rows (78.00 and 65.00, not below 50) were unchanged; and Rahul Sharma's city changed from Nashik to Mumbai.

DELETE (Screenshots 11–12): Before the delete Aman Verma (student_id 4) had two enrollments; `DELETE 1` removed the CS203 row and the verification `SELECT` shows only CS201 remaining. The total number of enrollments fell from 12 to 11. The attempt to delete student 4 from the *student* table failed with a foreign key violation on `enrollment_student_id_fkey`, because enrollment 9 still references that student – PostgreSQL protected referential integrity and no data was lost. The final enrollment listing shows all 11 rows with the updated values (rows 9 and 11) and without row 10.

9. Result

DML operations `INSERT`, `UPDATE` and `DELETE` were successfully performed on the College Management System database *college_db* in PostgreSQL 17. Data was retrieved using `SELECT`, filtered with `WHERE` (comparison, logical, `BETWEEN` and `LIKE` operators), sorted with `ORDER BY` and restricted with `LIMIT/OFFSET`. Every modification was verified with a `SELECT` statement, and the `FOREIGN KEY` constraints from P2 were observed to prevent deletion of a referenced student record.

10. Viva Questions and Answers

Q1. What is DML? Name its commands.

Ans: Data Manipulation Language is the part of SQL used to manage data inside tables. Its commands are `INSERT`, `UPDATE` and `DELETE` (and `MERGE`); `SELECT` is used to query the data.

Q2. What is the difference between DELETE, TRUNCATE and DROP?

Ans: `DELETE` is DML; it removes selected rows using `WHERE` and fires row-level triggers. `TRUNCATE` is DDL; it removes all rows quickly without scanning them and cannot use `WHERE`. `DROP` removes the table itself, including its structure, data and constraints.

Q3. What happens if the WHERE clause is omitted in UPDATE or DELETE?

Ans: The operation is applied to every row of the table – all rows are updated or all rows are deleted. Running the `WHERE` condition as a `SELECT` first is a safe habit.

Q4. Why is ORDER BY necessary when LIMIT is used?

Ans: Without `ORDER BY` the order of rows is not guaranteed, so `LIMIT` would return an arbitrary set of rows. With `ORDER BY` the result is deterministic, e.g. the top three marks.

Q5. What does OFFSET do?

Ans: OFFSET m skips the first m rows of the sorted result. LIMIT 3 OFFSET 3 returns rows 4 to 6 and is used for paging.

Q6. Is BETWEEN inclusive?

Ans: Yes. marks BETWEEN 60 AND 75 is equivalent to marks >= 60 AND marks <= 75.

Q7. Explain the wildcards used with LIKE.

Ans: % matches any sequence of zero or more characters and _ matches exactly one character. LIKE is case-sensitive in PostgreSQL; ILIKE performs a case-insensitive match.

Q8. What is the meaning of “INSERT 0 12” in psql?

Ans: It is the command tag returned by the server: 0 is the OID field (always 0 for tables without OIDs) and 12 is the number of rows inserted.

Q9. Why did DELETE FROM student WHERE student_id = 4 fail?

Ans: The enrollment table has a FOREIGN KEY on student_id referencing student. Enrollment 9 still referred to student 4, so deleting the parent would create an orphan row; PostgreSQL rejected it. It could succeed only if the child rows were deleted first or the key was defined with ON DELETE CASCADE.

Q10. What is the difference between WHERE and HAVING?

Ans: WHERE filters individual rows before grouping; HAVING filters groups after GROUP BY and can use aggregate functions such as COUNT or AVG.

Q11. How can a DML change be undone?

Ans: By running the statements inside a transaction: BEGIN; ... ROLLBACK; In psql's default autocommit mode, each statement is committed immediately and cannot be rolled back afterwards.

Q12. What is the RETURNING clause in PostgreSQL?

Ans: INSERT, UPDATE and DELETE can end with RETURNING column_list to return the affected rows, e.g. DELETE FROM enrollment WHERE ... RETURNING *; – useful to see exactly what was changed.

Q13. Why should column names be listed in an INSERT statement?

Ans: The statement then does not depend on the column order of the table, columns with defaults can be skipped, and it is easier to read and maintain.

Q14. Can one INSERT statement add many rows?

Ans: Yes, by writing several value lists separated by commas after VALUES, as done for all five tables in this practical. The statement is atomic: if any row violates a constraint, none of the rows are inserted.